

GUIA DE ESTUDOS

Python • resumo organizado e fácil de revisar

Como estudar

1. Leia o conceito.
2. Refaça os exemplos sem olhar.
3. Tente explicar com suas próprias palavras.
4. Revise os pontos-chave antes da prova.

1	Estruturas de dados	listas • tuplas • dicionários • conjuntos
2	Funções	def • parâmetros • return • lambda • escopo • docstrings • *args
3	Exceções	try • except • finally • raise
4	Entrada e saída	input • print • arquivos • with
5	Módulos e pacotes	import • from • módulos próprios • pacotes

listas

`append(elemento)`: adiciona um elemento ao final da lista.

`insert(indice, elemento)`: insere um elemento em uma posição específica da lista.

`remove(elemento)`: remove a primeira ocorrência de um elemento na lista.

`pop(indice)`: remove e retorna o elemento em uma posição específica da lista.

`sort()`: ordena os elementos da lista em ordem ascendente.

`reverse()`: inverte a ordem dos elementos na lista

Listas de compreensão

As listas de compreensão são uma forma concisa de criar novas listas baseadas em uma sequência existente. Permitem filtrar e transformar os elementos de uma lista em uma única linha de código.

```
nova_lista = [expressão for elemento in sequência if condição]
```

Exemplo:

números = [1, 2, 3, 4, 5]

```
quadrados = [x ** 2 for x in números if x % 2 == 0]  
print(quadrados) # Imprime [4, 16]
```

Neste exemplo, é criada uma nova lista chamada `quadrados`, que contém os quadrados dos números pares da lista `números`. A expressão `x ** 2` eleva cada elemento ao quadrado, e a condição `if x % 2 == 0` filtra apenas os números pares.

Métodos de tuplas

Embora as tuplas sejam imutáveis, Python fornece vários métodos úteis para trabalhar com elas:

`count(elemento)`: devolve o número de vezes que um elemento aparece na tupla.

`index(elemento)`: devolve o índice da primeira aparição de um elemento na tupla. Opcionalmente, pode-se especificar o início e fim da busca.

`len(tupla)`: embora não seja um método de tupla propriamente dito, esta função incorporada devolve o comprimento da tupla.

```
minha_tupla = (1, 2, 3, 2, 4, 2)  
print (minha_tupla.index(2)) # Saída: 1  
print (minha_tupla.index(2, 2)) #Saída: 3  
print (minha_tupla.index(2, 2, 4)) #Saída: 3
```

Métodos de dicionários

Os dicionários em Python têm vários métodos incorporados para manipular e acessar os elementos. Alguns métodos comuns são:

`keys()`: retorna uma visualização de todas as chaves do dicionário.

`values()`: retorna uma visualização de todos os valores do dicionário.

`items()`: retorna uma visualização de todos os pares chave-valor do dicionário.

`update(outro_dicionario)`: atualiza o dicionário com os pares chave-valor de outro dicionário.

Um conjunto é uma estrutura de dados mutável e não ordenada que permite armazenar uma coleção de elementos únicos. Os conjuntos são delimitados por chaves {} ou são criados utilizando a função set().

Criação e operações básicas

Para criar um conjunto, utilize chaves ou a função set():

```
frutas = {"maçã", "banana", "laranja"}
numeros = set([1, 2, 3, 4, 5])
```

Os conjuntos suportam operações matemáticas de conjuntos, como a união (|), a interseção (&), a diferença (-) e a diferença simétrica (^).

```
conjunto1 = {1, 2, 3}
conjunto2 = {3, 4, 5}
uniao = conjunto1 | conjunto2
print(uniao) # Imprime {1, 2, 3, 4, 5}
intersecao = conjunto1 & conjunto2
print(intersecao) # Imprime {3}
diferenca = conjunto1 - conjunto2
print(diferenca) # Imprime {1, 2}
diferenca_simetrica = conjunto1 ^ conjunto2
print(diferenca_simetrica) # Imprime {1, 2, 4, 5}
```

Métodos de conjuntos

Os conjuntos em Python têm vários métodos incorporados para manipular e acessar os elementos. Alguns métodos comuns são:

add(elemento): adiciona um elemento ao conjunto.

remove(elemento): remove um elemento do conjunto. Se o elemento não existir, gera um erro.

discard(elemento): remove um elemento do conjunto se estiver presente. Se o elemento não existir, não faz nada.

clear(): remove todos os elementos do conjunto.

Exemplo:

```
frutas = {"maçã", "banana", "laranja"}
```

```
frutas.add("pera")
```

```
print(frutas) # Imprime {"maçã", "banana", "laranja", "pera"}
```

```
frutas.remove("banana")
```

```
print(frutas) # Imprime {"maçã", "laranja", "pera"}
```

```
frutas.discard("uva")
```

```
print(frutas) # Imprime {"maçã", "laranja", "pera"}
```

```
frutas.clear()
```

```
print(frutas) # Imprime set()
```

As estruturas de dados em Python nos oferecem grande flexibilidade e potência para armazenar e manipular dados em nossos programas. As listas são úteis para coleções ordenadas e mutáveis, as tuplas para coleções ordenadas e imutáveis, os dicionários para armazenar pares de chave valor e os conjuntos para coleções não ordenadas de elementos únicos.

As funções são blocos de código reutilizáveis que nos permitem encapsular tarefas específicas e executá-las quando necessário. As funções nos ajudam a organizar nosso código, evitar a repetição e fazer com que nossos programas sejam mais modulares e fáceis de manter.

Definição e chamada de funções

Para definir uma função em Python, utilizamos a palavra-chave `def` seguida do nome da função e parênteses. Opcionalmente, podemos especificar parâmetros dentro dos parênteses. O bloco de código da função é indentado após os dois pontos.

Para chamar uma função, simplesmente escrevemos o nome da função seguido de parênteses:

```
def saudacao():  
    print("Olá, mundo!")
```

`saudacao()` # Imprime "Olá, mundo!"

Parâmetros e argumentos

As funções podem aceitar parâmetros, que são valores que são passados para a função quando ela é chamada. Os parâmetros são especificados dentro dos parênteses na definição da função.

```
def saudacao(nome):  
    print(f"Olá, {nome}!")
```

Ao chamar a função, fornecemos os argumentos correspondentes aos parâmetros:

`saudacao("João")` # Imprime "Olá, João!"

`saudacao("Maria")` # Imprime "Olá, Maria!"

Valores de retorno

As funções podem retornar valores usando a palavra-chave `return`. O valor de retorno pode ser usado pelo código que chama a função.

```
def soma(a, b):  
    return a + b  
resultado = soma(3, 4)  
print(resultado) # Imprime 7
```

Funções anônimas (lambda)

Python permite criar funções anônimas ou funções `lambda`, que são funções sem nome definidas em uma única linha. São comumente usadas para funções pequenas e concisas.

```
quadrado = lambda x: x ** 2  
print(quadrado(5)) # Imprime 25
```

Escopo das variáveis (local vs. global)

As variáveis definidas dentro de uma função têm um escopo local, o que significa que só são acessíveis dentro da função. Por outro lado, as variáveis definidas fora de qualquer função têm um escopo global e podem ser acessadas de qualquer parte do programa.

```
def funcao():  
    variavel_local = 10  
    print(variavel_local) # Acessível dentro da função  
variavel_global = 20  
def funcao2():  
    print(variavel_global) # Acessível de qualquer lugar
```

```
funcao() # Imprime 10
```

```
funcao2() # Imprime 20  
print(variavel_global) # Imprime 20  
print(variavel_local) # Gera um erro, a variável não está definida neste escopo.
```

Documentação de funções (docstrings)

É uma boa prática documentar nossas funções utilizando docstrings. Os docstrings são cadeias de texto que descrevem o propósito, os parâmetros e o valor de retorno de uma função. São colocados imediatamente após a definição da função e são encerrados entre aspas duplas triplas.

```
def area_retangulo(base, altura):
```

```
.....
```

Calcula a área de um retângulo.

Args:

base (float): A base do retângulo.

altura (float): A altura do retângulo.

Returns:

float: A área do retângulo.

```
.....
```

```
return base * altura
```

Funções com número variável de argumentos

Python permite definir funções que aceitem um número variável de argumentos. Isso é feito utilizando o operador * antes do nome do parâmetro.

```
def soma_variavel(*numeros):  
    total = 0  
    for numero in numeros:
```

```
total += numero
```

```
    return total  
print(soma_variavel(1, 2, 3)) # Imprime 6  
print(soma_variavel(4, 5, 6, 7)) # Imprime 22
```

As funções são uma ferramenta fundamental na programação e nos permitem estruturar e modularizar nosso código. Com a capacidade de definir funções personalizadas, podemos encapsular tarefas específicas e reutilizá-las em diferentes partes do nosso programa.

Além das funções definidas pelo usuário, Python também fornece uma ampla gama de funções incorporadas que podemos utilizar diretamente, como `print()`, `len()`, `range()`, entre outras.

O manejo de exceções nos permite capturar e lidar com erros de maneira controlada utilizando as declarações `try`, `except` e opcionalmente `finally`.

Try

O bloco `try` contém o código que pode gerar uma exceção. Se ocorrer uma exceção dentro do bloco `try`, o fluxo de execução é transferido para o bloco `except` correspondente.

```
try:  
    # Código que pode gerar uma exceção
```

```
resultado = 10 / 0 # Divisão por zero
print(resultado)
except ZeroDivisionError:
    print("Erro: Divisão por zero")
```

Except

O bloco `except` especifica o tipo de exceção que se deseja capturar e lidar. Você pode ter múltiplos blocos `except` para lidar com diferentes tipos de exceções.

```
try:
    # Código que pode gerar uma exceção
    resultado = 10 / 0 # Divisão por zero
    print(resultado)
except ZeroDivisionError:
    print("Erro: Divisão por zero")
except ValueError:
    print("Erro: Valor inválido")
```

Finally

O bloco `finally` é opcional e é executado sempre, independentemente de ter ocorrido uma exceção ou não. É comumente utilizado para realizar tarefas de limpeza ou liberação de recursos.

```
try:
    # Código que pode gerar uma exceção
    arquivo = open("arquivo.txt", "r")
    # Realizar operações com o arquivo
except FileNotFoundError:
    print("Erro: Arquivo não encontrado")
finally:
    arquivo.close() # Fechar o arquivo sempre, mesmo se ocorrer uma exceção
```

Além das exceções incorporadas no Python, você também pode criar suas próprias exceções personalizadas. Isso é útil quando deseja lidar com situações específicas do seu programa.

Para criar uma exceção personalizada, você deve criar uma classe que herde da classe base `Exception` ou de uma de suas subclasses.

```
def funcao():
    # Código que pode gerar uma exceção personalizada
    if condicao:
```

`raise Exception("Descrição do erro")`

```
try:
    funcao()
except Exception as e:
    print(f"Erro: {str(e)}")
```

Neste exemplo, define-se uma função chamada `funcao()`. Dentro da função, verifica-se uma condição `e`, se for satisfeita, gera-se uma exceção utilizando a declaração `raise`. Em vez de criar uma classe personalizada, utiliza-se diretamente a classe base `Exception` para gerar a exceção.

Depois, utiliza-se um bloco `try-except` para capturar e lidar com a exceção. A variável `e` é utilizada para acessar a descrição do erro fornecida ao gerar a exceção.

O tratamento de erros e exceções é uma parte fundamental da programação em Python. Permite lidar com situações inesperadas de maneira controlada e evitar que seu programa trave ou pare abruptamente.

Quando ocorre um erro no seu código, o Python gera uma exceção. Ao utilizar blocos `try-except`, você pode capturar e lidar com essas exceções de maneira adequada. Pode especificar diferentes blocos `except` para lidar

com diferentes tipos de exceções e realizar ações específicas em cada caso.

Além disso, o bloco `finally` permite executar código de limpeza ou liberação de recursos, independentemente de ter ocorrido uma exceção ou não. Isso é útil para garantir que certas ações sejam sempre realizadas, como fechar arquivos ou conexões de banco de dados.

IMPORTANTE

Considere os possíveis erros que podem ocorrer no seu código e utilize o tratamento de exceções adequado para lidar com eles de maneira apropriada. Isso tornará seus programas mais robustos e confiáveis.

Em Python, a entrada e saída de dados nos permite interagir com o usuário e manipular arquivos. Podemos solicitar informações ao usuário, mostrar resultados na tela e ler ou escrever dados em arquivos externos.

Entrada de dados do usuário

Para obter informações do usuário durante a execução do programa, podemos utilizar a função `input()`. Esta função mostra uma mensagem na tela e espera que o usuário insira um valor.

```
nome = input("Insira seu nome: ")
idade = input("Insira sua idade: ")
print("Olá, " + nome + "!")
print("Você tem " + idade + " anos.")
```

Neste exemplo, solicita-se ao usuário que insira seu nome e idade utilizando a função `input()`. Os valores inseridos são armazenados nas variáveis `nome` e `idade`, respectivamente. Em seguida, essas variáveis são utilizadas para mostrar uma saudação personalizada na tela.

IMPORTANTE

A função `input()` sempre retorna uma cadeia de texto. Se você deseja trabalhar com outros tipos de dados, como números inteiros ou flutuantes, deve realizar uma conversão explícita utilizando funções como `int()` ou `float()`.

Saída de dados

Para mostrar informações na tela, utilizamos a função `print()`. Esta função recebe um ou mais argumentos e os mostra no console.

Podemos utilizar a f-string (formatação de cadeias) para inserir variáveis diretamente dentro de uma cadeia de texto.

```
nome = "Juan"
idade = 25
print(f"Olá, meu nome é {nome} e tenho {idade} anos.")
```

Neste caso, as variáveis são inseridas dentro da cadeia utilizando chaves `{}` e a cadeia é precedida pela letra `f` para indicar que é uma f-string.

Python nos permite ler e escrever dados em arquivos externos. Podemos abrir arquivos em diferentes modos, como leitura ("`r`"), escrita ("`w`") ou anexar ("`a`"), e realizar operações de leitura e escrita.

Leitura de arquivos

Para ler o conteúdo de um arquivo, primeiro devemos abri-lo utilizando a função `open()` em modo de leitura ("`r`"). Depois, podemos ler o conteúdo do arquivo utilizando métodos como `read()` ou `readlines()`.

```
arquivo = open("dados.txt", "r")
conteudo = arquivo.read()
print(conteudo)

arquivo.close()
```

Neste exemplo, o arquivo "dados.txt" é aberto em modo de leitura utilizando open(). Depois, todo o conteúdo do arquivo é lido utilizando o método read() e armazenado na variável conteudo. Finalmente, o conteúdo é mostrado na tela e o arquivo é fechado utilizando o método close().

Escrita de arquivos

Para escrever dados em um arquivo, abrimos em modo de escrita ("w") utilizando a função open(). Se o arquivo não existir, será criado automaticamente. Se o arquivo já existir, seu conteúdo será sobrescrito.

```
arquivo = open("dados.txt", "w")
arquivo.write("Olá, mundo!")
arquivo.close()
```

Neste exemplo, o arquivo "dados.txt" é aberto em modo de escrita utilizando open(). Depois, a string "Olá, mundo!" é escrita no arquivo utilizando o método write(). Finalmente, o arquivo é fechado utilizando o método close().

IMPORTANTE

É importante fechar sempre os arquivos depois de utilizá-los para liberar os recursos do sistema.

Você também pode utilizar a declaração with para manejar a abertura e fechamento de arquivos de maneira automática.

```
with open("dados.txt", "r") as arquivo:
    conteudo = arquivo.read()
    print(conteudo)
```

Neste caso, o arquivo é aberto utilizando a declaração with e é fechado automaticamente uma vez que se sai do bloco with, mesmo se ocorrer uma exceção.

A entrada e saída de dados em Python nos oferece uma grande flexibilidade para interagir com o usuário e manipular arquivos externos. Podemos solicitar informações ao usuário, mostrar resultados na tela e ler ou escrever dados em arquivos de texto. Lembre-se sempre de manejar adequadamente a abertura e fechamento de arquivos, e considerar as possíveis exceções que podem ocorrer durante as operações de entrada/saída.

Em Python, um módulo é um arquivo que contém definições de funções, classes e variáveis que podem ser utilizadas em outros programas. A importação de módulos nos permite acessar a funcionalidade definida em outros arquivos e reutilizar código de maneira eficiente. Além disso, podemos criar nossos próprios módulos para organizar e modularizar nosso código.

SOA_LID_Ten en cuenta-1.png

Tenha em mente

Python vem com uma ampla biblioteca padrão de módulos que fornecem funcionalidades adicionais. Esses módulos estão disponíveis sem a necessidade de instalá-los separadamente.

Importar módulos

Para utilizar um módulo em nosso programa, devemos importá-lo utilizando a declaração import. Podemos importar um módulo completo ou funções específicas de um módulo.

```
import math
resultado = math.sqrt(25)
print(resultado) # Imprime 5.0
```

Neste exemplo, importa-se o módulo math utilizando a declaração import. Em seguida, utiliza-se a função sqrt() do módulo math para calcular a raiz quadrada de 25.

Também podemos importar funções específicas de um módulo utilizando a sintaxe from módulo import função.


```
from math import sqrt
resultado = sqrt(25)
print(resultado) # Imprime 5.0
```

Neste caso, importa-se apenas a função `sqrt()` do módulo `math`, o que nos permite utilizá-la diretamente sem ter que precedê-la com o nome do módulo.

Funções e classes de módulos padrão

A biblioteca padrão de Python oferece uma ampla gama de módulos com funções e classes úteis. Alguns exemplos comuns incluem:

```
import random
import datetime
numero_aleatorio = random.randint(1, 10)
print(numero_aleatorio) # Imprime um número inteiro aleatório entre 1 e 10
data_atual = datetime.datetime.now()
print(data_atual) # Imprime a data e hora atual
```

Estes são apenas alguns exemplos dos muitos módulos disponíveis na biblioteca padrão de Python. Você pode consultar a documentação oficial de Python para obter mais informações sobre os módulos e suas funcionalidades.

Além de utilizar os módulos padrão do Python, também podemos criar nossos próprios módulos para organizar e reutilizar nosso código.

Criar e utilizar módulos personalizados

Para criar um módulo personalizado, simplesmente criamos um novo arquivo Python com o nome desejado e definimos as funções, classes e variáveis que queremos incluir no módulo. Por exemplo, criamos um arquivo (no mesmo diretório onde estamos executando Python) chamado `meu_modulo.py` com o seguinte conteúdo:

```
#meu_modulo.py
def saudar(nome):
    print(f"Olá, {nome}!")
def calcular_soma(a, b):
    return a + b
```

Depois, podemos importar e utilizar as funções definidas em `meu_modulo.py` em outro arquivo Python.

```
import meu_modulo

meu_modulo.saudar("João") # Imprime "Olá, João!"
resultado = meu_modulo.calcular_soma(5, 3)
print(resultado) # Imprime 8
```

Neste exemplo, importa-se o módulo `meu_modulo` e utilizam-se as funções `saudar()` e `calcular_soma()` definidas nele.

Organização do código em módulos

À medida que nossos programas crescem em tamanho e complexidade, é uma boa prática organizar nosso código em módulos separados segundo sua funcionalidade. Isso nos permite manter um código mais legível, agrupado em módulos e fácil de manter.

Por exemplo, podemos ter um módulo `operacoes.py` que contenha funções relacionadas com operações matemáticas, e outro módulo `utilidades.py` que contenha funções de uso geral.

```
# operacoes.py
def somar(a, b):
    return a + b
def subtrair(a, b):
    return a - b
```

```
# utilidades.py
def imprimir_mensagem(mensagem):
    print(mensagem)
def obter_nome_usuario():
    return input("Digite seu nome: ")
```

Depois, podemos importar e utilizar essas funções em nosso programa principal.

```
import operacoes
import utilidades
resultado = operacoes.somar(5, 3)
```

utilidades.imprimir_mensagem(f"O resultado da soma é: {resultado}")

```
nome = utilidades.obter_nome_usuario()
```

utilidades.imprimir_mensagem(f"Olá, {nome}!")

Ao organizar nosso código em módulos, podemos reutilizar funções e manter um código mais estruturado e agrupado em módulos.

Um pacote é uma forma de organizar módulos relacionados em uma estrutura hierárquica de diretórios. Os pacotes nos permitem agrupar módulos relacionados e evitar conflitos de nomes entre módulos.

Criar e utilizar pacotes

Para criar um pacote, criamos um diretório com o nome desejado e adicionamos um arquivo especial chamado `__init__.py` dentro do diretório. Este arquivo pode estar vazio ou conter código de inicialização do pacote.

Por exemplo, criamos um diretório chamado `meu_pacote` com a seguinte estrutura:

meu_pacote/

__init__.py

```
modulo1.py
```

```
modulo2.py
```

Depois, podemos importar e utilizar os módulos do pacote em nosso programa.

```
from meu_pacote import modulo1, modulo2

modulo1.funcao1()

modulo2.funcao2()
```

Neste exemplo, são importados os módulos `modulo1` e `modulo2` do pacote `meu_pacote` e são utilizadas as funções definidas neles.

A importação e criação de módulos e pacotes em Python nos permite organizar e reutilizar nosso código de maneira eficiente. Ao modularizar nosso código, podemos manter um código mais legível, estruturado e fácil de manter.

Lembre-se de explorar a biblioteca padrão de Python e aproveitar os módulos existentes, que podem facilitar muitas tarefas comuns. Além disso, não hesite em criar seus próprios módulos e pacotes para organizar e reutilizar seu código de maneira eficaz.